SAPIENZA
UNIVERSITÀ DI ROMA

DEPARTMENT OF INGEGNERIA INFORMATICA,
AUTOMATICA E GESTIONALE

A Spider in Space

INTERACTIVE GRAPHICS

Professors:
Marco Schaerf

Students:
Giacomo Ceccarini - 2216506
Francesco Maria Germano - 2212589

Academic Year 2025/2026

1 Introduction

A Spider in Space is a browser-based arcade game in which the player drives a large mechanical spider across the surface of a tiny moon, collecting glowing crystals against a running-down clock. The spider is not driven by any pre-recorded animation: every leg movement is computed in real time by an **inverse-kinematics solver**, the body sway is produced by a physical spring, and the whole world is bent into a miniature planet by a custom **vertex shader**. The visible planet in the sky and the surrounding galaxy are provided by an **equirectangular skybox**, so the moon appears to orbit a larger body exactly as the concept intends.

The project deliberately puts its weight on the technical requirement that carries the most credit in this course: **all animation is implemented from scratch in JavaScript**. The spider model is imported as a rigged mesh (which the assignment allows) but no animation track is ever imported. The eight-legged walk cycle, the body dynamics, the crystal bobbing and the camera moves are all generated procedurally at run time.

1.1 Concept and design pillars

- **Procedural locomotion.** The spider walks with a custom **FABRIK** solver on all eight legs, a foot-locking gait, and a spring-damped pelvis for secondary motion.
- **A world that is flat to the code but round to the eye.** A "curved world" shader bends the ground into a horizon, so the player feels they are standing on a small sphere while every gameplay computation stays on a simple plane.
- **A seamless, borderless surface.** The level wraps toroidally: walk far enough in any direction and you return from the opposite side, with no visible edge.
- **Readable, self-contained gameplay.** A countdown timer, three distinct crystal types, a stamina-limited sprint and an HTML HUD form a small but complete game loop.

2 Development Environment

The project is written in **vanilla JavaScript (ES modules)** and rendered with **Three.js**, a high-level WebGL library. The choice of Three.js over raw WebGL is explicitly permitted by the assignment and lets the team concentrate on the parts that carry the technical value of the project (the inverse kinematics, the gait, and the shaders) rather than on boilerplate such as buffer management and matrix plumbing.

- **Vite** was used as the development server and build tool.
- **lil-gui** provides an on-screen control panel used throughout development to tune every parameter live (gait timings, spring stiffness, curvature radius, lighting, and so on). The panel is a development aid and is not part of the intended play experience.
- **No physics engine is used.** The assignment lists physics engines as optional. Because the spider is animated kinematically and the gameplay is based on simple proximity tests, a physics engine would have added weight and complexity without improving the result, so none was integrated.

3 Libraries, Tools and Assets Not Developed by the Team

The following external components are used. Everything else (all game logic, the IK solver, the gait, the shader, the UI) was written by the team.

3.1 Libraries and tools

Component	Role	Source / License
Three.js	WebGL rendering library (scene graph, materials, loaders)	`threejs.org` — MIT
Vite	Development server and production bundler	`vitejs.dev` — MIT
lil-gui	Live parameter panel (development aid)	`github.com/georgealways/lil-gui` — MIT
FBXLoader, GLTFLoader, OrbitControls (Three.js add-ons)	Model import and mouse camera	Three.js examples/jsm — MIT

3.2 Models

Model	Use	Source / License
CreepySpider skeletal mesh	The player character (rigged FBX, no animation imported)	Team's own bachelor-thesis asset — see note below
Crystal mesh	The collectible crystal (single mesh, reused for all three crystal types)	Fab — Standard License

Note on the spider. The spider mesh and its skeleton originate from a previous bachelor-thesis project by a team member (a procedural walk-cycle study in Unreal Engine). It is listed here for full transparency as pre-existing material. Only the **geometry, skeleton and textures** were reused; the entire animation system in this project was rebuilt from scratch in JavaScript.

3.3 Textures

Texture set	Use	Source / License
Spider maps (color, normal, specular, ambient occlusion)	Spider surface material	From the thesis asset (Unreal/Quixel)
Gravel Ground + 2 further ground sets (color, normal, specular, height)	The three selectable ground surfaces	Quixel Megascans via Fab — Standard License
"Orbital 10 – Sunset" equirectangular panorama (+ 5 further skies)	Skybox (planet + space)	RenderCrate — used as tone-mapped JPG
Crystal maps (base color, normal, roughness, emissive)	Crystal surface material	From the crystal asset package (Fab)

3.4 Audio

A looping soundtrack and a set of short sound effects (one per crystal type, two for the risk crystal, plus menu navigation and confirmation sounds) are the project's tracks. Files: `ost.mp3`, `pickup_normal/bonus/risk_good/risk_bad.mp3`, `menu_sound.mp3`, `menu_play.mp3`.

4 Compliance with the Course Requirements

The table below maps each mandatory requirement to its implementation in the project.

Requirement	How it is satisfied
Hierarchical model (at least one complex)	The rigged spider skeleton: eight identical leg chains (thigh → calf → foot) beneath a common pelvis/neck/head hierarchy: 28 bones in total.
Lights	Two lights: a directional "sun" providing uniform parallel illumination across the whole moon, and a weak ambient fill that keeps the side of the spider facing away from the sun readable. Shadow mapping is intentionally omitted.
Textures of different kinds	Color, normal, specular and ambient-occlusion maps on the spider; color, normal, specular and height maps on the ground; an equirectangular skybox and emissive maps on the crystals.
User interaction	Keyboard movement (forward/back, strafe, turn), stamina-limited sprint, camera toggle and free mouse-orbit, pause, and a keyboard-driven menu that selects difficulty, ground texture, skybox and volume.
Animations implemented in JavaScript	FABRIK inverse kinematics on all eight legs, a foot-locking gait, a spring-damped pelvis, crystal bobbing and spin, body tilt and cinematic camera moves. No animation track is imported.
Delivery	All source and libraries in the repository; GitHub Pages build; this accompanying document as technical report and user manual.

5 Technical Aspects

5.1 Application architecture and per-frame order

The spider is represented by three transform layers and they never interfere with each other. The outermost is a logical root (an empty `THREE.Group`) which carries the spider's position and heading: all movement and turning is written here; the imported model is a child of the root and receives only a cosmetic pitch/roll tilt. Finally, inside the model, the individual bones of the skeleton receive the procedural animation: the pelvis bone takes the bob and sway offsets, and the leg bones are posed by the IK solver.

Because the legs are solved last, they automatically absorb the body sway, the tilt and the movement: the feet stay planted while everything above them moves. The per-frame order is therefore essential and is fixed as follows:

```
read input  ->  move / rotate the root  ->  tilt the model
            ->  wrap the world (toroidal teleport)
            ->  update the game (crystals, timer, HUD)
            ->  measure the root velocity
            ->  update the pelvis spring
            ->  updateMatrixWorld()
            ->  update the gait  ->  solve FABRIK on the 8 legs
            ->  update the camera  ->  render
```

5.2 Model loading and skeleton discovery

The spider is loaded with `FBXLoader`. We had one major issue to be handled. Scale: Unreal exports in centimetres, so the raw mesh is enormous; the model is auto-normalized at load to a fixed leg span with `fbx.scale.setScalar(2.5 / max(size.x, size.z))`, then dropped onto the ground plane via its bounding box.

A subtlety discovered by inspecting the hierarchy is that the eight legs are children of the `head` bone, not of the pelvis directly: `Pelvis` → `neck` → `head` → { 8 × (`thigh` → `calf` → `foot`) }. This is harmless for the IK (each chain is solved from its own thigh) and is in fact exploited by the pelvis animation, since moving the pelvis moves the whole body and the legs compensate.

5.3 Inverse kinematics: the FABRIK solver

Each leg is a two-segment chain `[thigh, calf, foot]` whose foot acts as the end effector. The legs are posed by FABRIK (Forward And Backward Reaching Inverse Kinematics, Aristidou & Lasenby, 2011), a fast iterative solver that operates directly on joint positions rather than on angles. Given a target, it repeats two passes until

the end effector is close enough: a backward pass that pulls the chain from the end effector back to the root, and a forward pass that pushes it from the root back out to the effector, each pass preserving the fixed segment lengths.

```
// one FABRIK iteration (positions p[], fixed lengths[])
p[last] = target;
for (i = last-1 .. 0)  p[i] = p[i+1] + dir(p[i]-p[i+1]) * len[i];   // backward
p[0] = root;
for (i = 1 .. last)    p[i] = p[i-1] + dir(p[i]-p[i-1]) * len[i-1]; // forward
```

When the target is farther than the chain can reach, the solver falls back to laying the chain out straight toward the target. This behavior is correct, but it introduces a geometric problem addressed next.

5.3.1 The pole constraint

Once thigh, calf and foot become collinear (which happens whenever a leg stretches fully) FABRIK loses the information about *which side* the knee should bend toward. This is a genuine mathematical singularity, and without a remedy the knee can flip to a random side for a frame. The remedy is a pole constraint: after each solve, the middle joint (the knee) is rotated around the axis from shoulder to foot until it points toward a pole placed above the leg. Rotating the knee around the shoulder-to-foot axis moves it along a circle without changing its distance from either end, so the bone lengths stay correct: the leg is only being oriented, not stretched. And because "up" is recomputed from scratch each frame from the current shoulder and foot positions, the knee therefore always bends upward and never flips, even immediately after a full stretch.

5.3.2 Turning positions into bone rotations (retargeting)

FABRIK works entirely with *positions*: it decides where each joint should be as a point in space. A skeleton, however, is not controlled by moving points: each bone is controlled by a *rotation*. There is therefore a gap to bridge: the solved joint positions must be translated into bone rotations that place the joints there. This translation is called *retargeting*, and it is the same step any IK system performs: the solver decides where the limb should reach, then the rig's bones must be rotated to match.

The routine does this one bone at a time. For each bone it compares the direction the bone currently points (from the bone toward its child joint) with the direction it *should* point (toward the position FABRIK computed for that child) and builds the rotation that carries the first direction onto the second: in Three.js a single call, `setFromUnitVectors()`.

5.4 The gait controller

The IK poses a leg to reach a target; the gait decides **where and when** each target moves. Its design rests on a few ideas that together produce a stable, natural walk.

- **Home positions in body space.** Each leg stores its rest foot position in the body's local frame and re-projects it to world space every frame, so the "home" a foot wants to occupy translates and rotates with the body.
- **Foot locking.** While a foot is grounded, its IK target equals a fixed planted world position. The foot literally does not move until it decides to step, which eliminates sliding by construction.
- **Step trigger.** A grounded foot lifts when the horizontal distance between its home and its planted position exceeds a threshold.
- **Alternating tetrapod groups.** The eight legs are split into two diagonal groups. Turns are assigned per group with strict alternation (after group A steps, group B is next) which guarantees that at least four feet are always on the ground and ensures no group can ever get stuck waiting forever without stepping (a real problem in an earlier version, where one set of legs kept stealing every turn and the other stayed frozen to the ground).
- **Body-anchored landing.** The spot where a lifted foot will land is stored relative to the body rather than as an absolute point in the world, so it moves together with the root (it translates and rotates with the spider automatically). No prediction is needed: the target simply follows the body because it is attached to the body's own frame of reference.

This approach replaced an earlier, more complicated one. The first attempt tried to *predict* where the foot should land from the body's velocity plus an extra term for its rotation. It was fragile, especially during turns, and was ultimately scrapped. Anchoring the landing spot to the body instead gives the same "the foot lands where the body is heading" result for free, because the body's movement and rotation are already built into its transform: the target moves with it, so there is nothing to predict.

Step frequency is never set directly; it *emerges* from the ratio of walking speed to step threshold, and the step duration is kept below that ratio so a group always finishes its step before the other exceeds threshold. During sprint the step duration is shortened, producing a visibly more frantic, faster gait.

5.5 Secondary animation: the pelvis spring

To give the spider weight and character, the `Pelvis` bone is driven by a **semi-implicit spring-damper**. A target offset is computed each frame (a vertical bob (at twice the

gait frequency, matching the two support beats per cycle), a lateral sway, a "lift" that raises the body while moving, and a slow idle "breathing" at rest) and the spring chases that target with a small, deliberately under-damped overshoot.

The intensity of the motion is not switched on and off; it is **ramped**. When the player stops, the target relaxes toward rest and the under-damped spring settles with a gentle bounce: the "return-to-idle" animation therefore emerges from the physics rather than being scripted, and a low breathing oscillation keeps the spider alive when standing still. A continuous cross-fade between the "moving" and "idle" targets means there is no discontinuity at any intermediate speed.

5.6 The curved-world shader

The signature look (standing on a small planet whose ground curves away in every direction) is produced by a vertex shader injected into the ground material through Three.js's `onBeforeCompile` hook. Every ground vertex is displaced downward in proportion to the square of its horizontal distance from the spider:

```
vec2 d = worldPos.xz - uSpiderPos;
worldPos.y -= dot(d, d) / uCurveR;   // tiny-planet curvature
```

The effect is **purely visual**: the game logic (IK, gait, collisions, wrapping) all stays on a flat plane. Only the rendered geometry bends. The strength of the effect is set by a curvature-radius parameter: a smaller radius produces a smaller, more sharply curved moon. During development this was tuned live and then fixed to a single value for the final build. A useful side effect of the curvature is that, since the ground bends below the line of sight, the far edge of the plane is hidden from view entirely: the curvature acts as a natural horizon, so no artificial fog or wall is needed to conceal the boundary of the world.

The same shader also adds rolling hills to the distant terrain, while keeping the ground flat directly under the spider. It does this with a simple distance test: for each vertex, a height sampled from the ground's height map is added to its elevation, but only when the vertex is far enough from the spider. Within a fixed radius around the spider the added height is zero (the terrain stays perfectly flat, so the feet never walk over a bump); beyond that radius it fades smoothly up to full strength, producing hills on the horizon. Because the spider's position is fed to the shader every frame, this flat circle travels with the spider.

The height is sampled so that it repeats with a period exactly equal to the toroidal cell size. This matters because the world wraps: when the spider is teleported from one edge of the cell to the opposite one, the two edges are the same place in the wrapped world, so the hills must look identical on both sides. Repeating the height pattern at the cell period guarantees that the distant terrain matches perfectly across a wrap,

and the teleport stays invisible. The curvature is applied to the crystals as well, so they bend with the world instead of floating above it (the two shader variants, with and without hills, are cached separately so they never interfere).

5.7 The toroidal (borderless) world

The moon has no walls: walking off one edge brings the player back from the opposite side. Rather than build a real sphere (which would have meant re-orienting gravity and rewriting every "up is Y" assumption in the IK, the gait, the pelvis and the shader) the world is a flat **toroidal cell** of side 40. When the spider passes the cell boundary it is teleported to the opposite side. The teleport is invisible for two reasons: the curved-world shader has already bent the terrain below the horizon long before the edge, and the ground texture tiles at a period that divides the cell size, so the surroundings are pixel-identical before and after the jump.

The one delicate point is that foot-locking lives in world space. If only the root were teleported, the planted feet would be left behind and the legs would snap. The solution is to shift **all world-space state** by the same offset in a single operation (planted feet, IK targets, the swing endpoints, the previous position used for velocity, the camera and the orbit target) while the body-anchored quantities are left untouched because they already move with the body.

5.7.1 Rendering the crystals across the wrap

The rule is simply: keep each crystal's logical position fixed, and every frame draw it at its nearest repetition to the spider. This "nearest repetition" is found by wrapping the crystal-to-spider distance back into the $[-20, +20]$ range: the same idea as reading a clock, where 23:00 is better expressed as "one hour ago" than "twenty-three hours from now". (In code this wrapping uses a modulo applied twice, only to make the arithmetic behave for negative coordinates.)

5.8 Materials, lighting and the skybox

Every surface uses `MeshPhongMaterial`, i.e. the **Blinn-Phong** illumination model taught in the course: ambient plus diffuse plus a specular highlight whose tightness is controlled by the `shininess` parameter. The spider carries four texture types (color, normal, specular, ambient occlusion); the ground carries color, normal, specular and height. Normal maps exported in the DirectX convention (Quixel) are corrected with `normalScale.y = -1`, otherwise the relief would appear inverted with bumps reading as dents.

Lighting is intentionally simple and readable: a single **directional light** acts as a fixed sun, illuminating the entire moon uniformly with parallel rays and no distance

falloff, and a weak ambient light keeps the shadows from turning pure black. The **skybox** is an equirectangular panorama assigned to `scene.background`; its orientation is set with `scene.backgroundRotation`, which allows the planet to be placed high overhead so it "looms" above the player and never competes with the curved horizon below. Ground texture and skybox can be switched at run time, which is what powers the level selection.

5.9 The camera system

Two camera modes are available and toggled live. The default is a **follow camera** that trails the spider from behind and above, easing toward its goal with an exponential lag so that motion feels smooth rather than rigid. The alternative is a **free orbit camera** (Three.js `OrbitControls`) that lets the player rotate and zoom around the spider with the mouse while it continues to walk. Switching modes preserves the wrap behaviour, since the orbit target is shifted together with the camera on every teleport.

5.10 Gameplay systems

The game is a race against a **countdown timer**. Time starts at 63, 48 or 33 seconds depending on the chosen difficulty and decreases every frame; reaching zero ends the run. Collecting crystals adds time. Three crystal types create the risk-reward feeling of the loop:

- **Normal crystal**: the common type, worth a modest time bonus.
- **Bonus crystal**: rarer and worth more; at most one exists at a time.
- **Risk crystal**: a gamble: collecting it is a 50/50 chance of a large time gain or a time loss; at most two exist at a time. A bad outcome can end the run.

Up to six crystals exist simultaneously. Their types are chosen by a **weighted draw** whose weights are renormalized if a specific type reaches its maximum, and new crystals spawn at a minimum distance both from the spider and from each other, so they never overlap or appear on top of the player. Each crystal **bobs** on a sine wave and slowly **spins**: both procedural JavaScript animations. Collection is a simple proximity test against the wrap-aware crystal position; on pickup, the time delta is applied, the appropriate sound plays, and a floating "+N / −N" number rises next to the timer.

5.11 Input and movement

Movement combines a forward/back axis and a strafe axis into a single direction that is normalized so that diagonal motion is not faster than straight motion. Turning

rotates the root. A **stamina-limited sprint** multiplies the speed while `Shift` is held: stamina drains during sprint, regenerates when not sprinting, and after being fully depleted enters a short cool-down before it can be used again, preventing rapid on/off tapping. Movement also drives a cosmetic body tilt (the spider pitches forward as it accelerates and rolls into turns and strafes) which is damped so the tilt eases in and out.

5.12 Audio

A looping soundtrack plays during the game, and short sound effects mark each event: a distinct sound per crystal type, two separate sounds for the risk crystal's good and bad outcomes, and dedicated navigation and confirmation sounds in the menu. A **master volume** scales both music and effects and is adjustable from the menu in steps of five. Because browsers block audio autoplay, the soundtrack is started from a genuine user gesture: the Play button.

5.13 User interface: HUD and menus

The **HUD** is drawn in HTML above the WebGL canvas: a large countdown timer that turns red in the final seconds, a crystal counter, a stamina bar that recolours when exhausted, the floating time-delta numbers on pickup, and a game-over screen with a restart button.

The **main menu** is fully keyboard-driven. The player moves between rows with the arrow keys, changes a value with left/right, and confirms Play with `Enter`. Difficulty, master volume, ground texture and skybox are all adjusted here, and ground and sky update **live** so the galaxy visibly changes as the player scrolls. The **pause menu** is opened and closed with `Esc`: it covers the scene with a dark semi-transparent overlay, shows a large "PAUSE" title and a panel listing the controls. While paused the frame time is consumed and discarded so that, on resume, the simulation does not jump forward by the paused duration.

6 Implemented Interactions

The project implements the following interactions, spanning direct character control, camera control, and menu-driven configuration.

Control	**Effect**
`W / S`	Move the spider forward / backward
`A / D`	Strafe left / right
`Q / E`	Turn (rotate) left / right
`Shift`	Sprint: a faster speed limited by a regenerating stamina bar
`C`	Toggle between follow camera and free mouse-orbit camera
`Mouse`	In orbit mode, rotate and zoom the camera around the spider
`Esc`	Pause / resume (dimmed overlay with the controls panel)
`Menu` — arrows	Navigate rows (up/down) and change values (left/right)
`Menu` — Enter	Confirm Play and trigger the cinematic intro

In addition to the explicit controls, the game reacts continuously to the player: the gait adapts its rhythm to speed and to turning, the body tilts with acceleration, crystals are collected on proximity, and the difficulty, ground texture, skybox and volume chosen in the menu reconfigure the session.

7 User Manual

7.1 Running the game

Online. Open the GitHub Pages link listed on the title page in a desktop browser. No installation is required.

Locally. From the project folder, install dependencies and start the development server:

```
npm install
npm run dev        # then open the printed http://localhost address
```

7.2 The main menu

The game opens on a view of the sky. Use the menu as follows:

- **Up / Down arrows**: move the highlight between the rows: Play, Difficulty, Volume, World texture, Skybox.
- **Left / Right arrows**: change the highlighted value. Difficulty cycles Easy / Normal / Hard (more time / less time); Volume changes in steps of five; World texture and Skybox cycle through the three available sets, and you will see the sky change immediately.
- **Enter**: with Play highlighted, start the game. The camera then descends from the sky onto the spider; control is handed to you once it settles, and the timer begins.

7.3 How to play

You control the spider on the surface of a small moon. Glowing crystals float across the surface; walk into a crystal to collect it and add time to the clock at the top of the screen. The clock is always counting down: if it reaches zero, the run is over.

The three crystal colours behave differently:

- **Common crystals** give a small, reliable time bonus.
- **The rarer bonus crystal** gives more time and replenishes stamina, grab it when you see it.
- **The grey risk crystal** is a gamble: it may give a large amount of time or take some away. Collect it when you are comfortable or avoid it when your clock is low.

Hold `Shift` to **sprint** toward a distant crystal, but watch the stamina bar at the bottom: it empties as you sprint and needs a moment to refill. Because the moon is round and borderless, you never hit a wall: keep moving in any direction and the surface simply wraps around. Press `C` at any time to switch to the free camera and look around with the mouse, and `Esc` to pause.

7.4 Tips

- Plan a route between several crystals rather than chasing the nearest one, the clock rewards efficient paths.
- Save sprint for when a crystal is about to drift out of reach or when the clock is dangerously low.
- Only take the risk crystal when a bad outcome would not end your run.

8 Conclusion

A Spider in Space brings together a set of techniques that reinforce one another: an inverse-kinematics solver and a foot-locking gait give the spider a believable, entirely procedural walk; a spring-damped pelvis adds weight and life; a curved-world shader and a toroidal topology create an endless miniature planet while keeping every computation flat and cheap; and a compact game loop of timers, crystal types and a stamina sprint turns the technology demonstration into something playable. Throughout, the project honours the central rule of the assignment of having **every animation generated in JavaScript, and none imported** while using Three.js, the Blinn-Phong model and a small set of external assets to keep the focus on the graphics work that carries the value of the project.